
DSC 140B - Homework 07

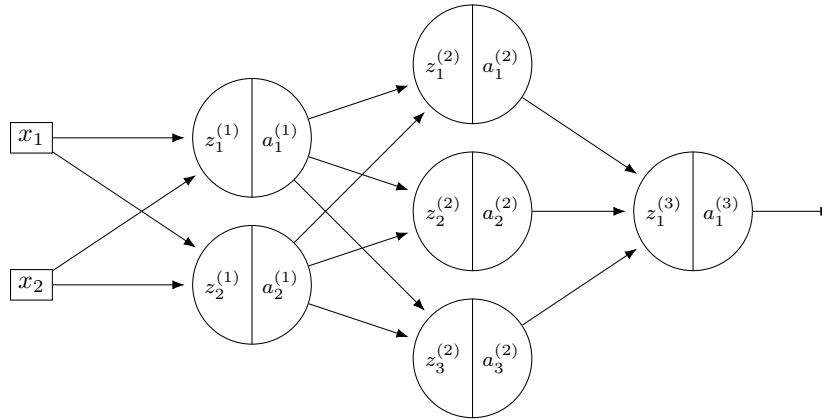
Due: Wednesday, February 25

Instructions:

- Write your solutions to the following problems **by hand**, either on another piece of paper that you scan or using a tablet. Typed solutions will not be accepted for credit!
 - Code listings are an exception. You do not need to handwrite code, and you can instead include the code as a screenshot.
- Unless otherwise noted by the problem's instructions, show your work or provide some justification for your answer.
- Homework problems are graded pass/fail on completeness and effort, not correctness.
- Homeworks are due via Gradescope at 11:59 PM.

Problem 1. (1 credit)

Consider the neural network H shown below. You may assume for simplicity that there are no bias weights. Assume that the hidden layers use the ReLU as their activation function, and that the output layer uses the linear activation.



Suppose that $\vec{x} = (2, 1)^T$ and that the weights of this network are:

$$W^{(1)} = \begin{pmatrix} 2 & -2 \\ 3 & 1 \end{pmatrix} \quad W^{(2)} = \begin{pmatrix} 2 & 1 & 2 \\ -3 & -1 & 0 \end{pmatrix} \quad W^{(3)} = \begin{pmatrix} 4 \\ 1 \\ 2 \end{pmatrix}$$

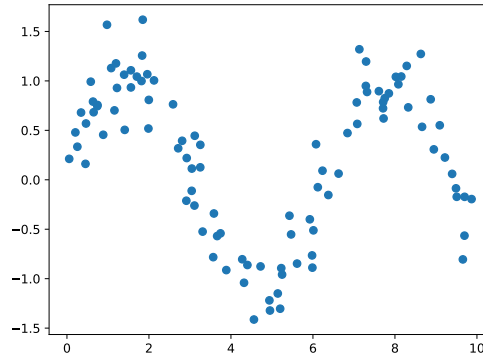
Carry out backpropagation in order to fill in the tables of partial derivatives below. Each entry should be a number. Show your work.

Hint: some of the entries of the tables have been filled in for you – use these to check your work.

		$\frac{\partial H}{\partial W_{11}^{(2)}}$	7		
		$\frac{\partial H}{\partial W_{12}^{(2)}}$			
		$\frac{\partial H}{\partial W_{13}^{(2)}}$			
		$\frac{\partial H}{\partial W_{21}^{(2)}}$			
		$\frac{\partial H}{\partial W_{22}^{(2)}}$			
		$\frac{\partial H}{\partial W_{23}^{(2)}}$			
				$\frac{\partial H}{\partial W_{11}^{(3)}}$	14
				$\frac{\partial H}{\partial W_{21}^{(3)}}$	
				$\frac{\partial H}{\partial W_{31}^{(3)}}$	

Problem 2. (1 credit)

In a previous homework, you trained a Gaussian RBF regression model on the data shown below:



You find this data at:

https://f000.backblazeb2.com/file/jeldridge-data/008-noisy_sin/data.csv

Now, train a neural network H on the same data. The architecture of the network is up to you, and you may decide how many hidden layers and hidden units to use. The network should be a feed-forward, fully-connected network that takes a single real number as input and produces a single real number as output. The loss function, activation functions, and optimization algorithm are also up to you.

Your model should be able to achieve an MSE of less than 0.15 on the training data.

- What architecture (number of hidden layers and nodes) did you choose for your network? How did you choose this architecture?
- What activation functions did you use on the hidden layers and the output layer? Why?
- Which loss function did you use to train the network? Why?
- Plot the predictions of your trained network in the interval $[0, 10]$, overlaid with the original data.
- Plot the empirical risk of your network as a function of the training iteration.
- Provide the code you used to train your network.

Problem 3. (2 credits)

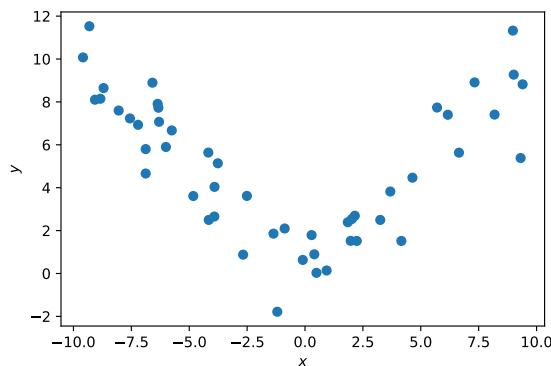
In this problem, you will train a simple deep neural network “from scratch” using only `numpy`.

This problem will make use of the following data set for regression:

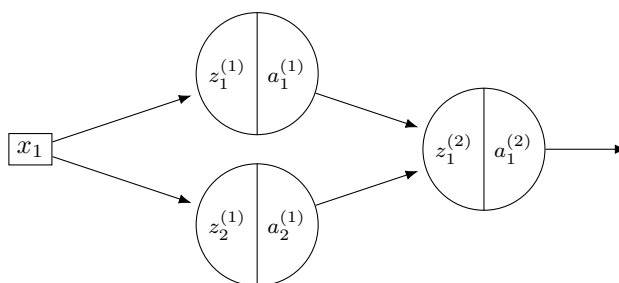
https://f000.backblazeb2.com/file/jeldridge-data/009-noisy_abs/data.csv

Each line in that file contains a training point; the first number being the feature, x , and the second being the target, y .

When plotted, the data looks like a noisy absolute value function:



The task is to train a neural network $H(x)$ to approximate this function. The architecture that our network will use is very simple:



We will assume that the hidden nodes use the ReLU, while the output node uses a linear activation, and that all nodes include a bias. This means that the network has seven parameters in total.

In what follows, you will be writing code that, for convenience, accepts all seven parameters as a *parameter vector* $\vec{w}$ with seven entries:¹

$$\vec{w} = (W_{00}^{(1)}, W_{01}^{(1)}, b_0^{(1)}, b_1^{(1)}, W_{00}^{(2)}, W_{10}^{(2)}, b_0^{(2)})^T$$

You might sometimes find it more convenient to work with the matrices $W^{(1)}$ and $W^{(2)}$, as well as the bias vectors $\vec{b}^{(1)}$ and $\vec{b}^{(2)}$. Therefore, we’ve provided a helper function, `weights_and_biases`, that will “convert” a parameter vector to this alternative form. It is available in the following GitHub Gist:

<https://gist.github.com/eldridgejm/4acce839c661f93412860f437f2a244e>

- a) Write a function, `H(x, w)`, which takes in a number, x , and a parameter vector w , and returns the result of evaluating the neural network at x with the parameters w . Use your function to compute $H(5, (1, 2, -1, -2, 3, 4, -5)^T)$. Show your code. Remember, you can use `numpy` in this problem, but you shouldn’t use any other libraries (e.g., `pytorch` or `tensorflow`).

¹Note that the indices on W and b start counting from zero, since `numpy` arrays are zero-indexed.

Hint 1: `H` should return a single number, but you might want to write a helper function which returns all of the a_{ij}^ℓ 's and $z_{ij}^{(\ell)}$'s as well, as they will be useful in the next problem and you need to compute them to compute $H(x)$ anyways.

Hint 2: You can check your answer by evaluating the network by hand.

Hint 3: Your answer should be a two-digit integer whose digits add to 12.

- b) Write a function `del_H(x, w)` that takes in a number, `x`, and a parameter vector `w`, performs backpropagation to compute $\nabla H(x, \vec{w})$, and returns the result as a numpy array with 7 entries. Using your function, compute $\nabla H(5, (1, 2, -1, -2, 3, 4, -5)^T)$. Show your code.

Hint 1: Don't worry about implementing backprop in its most general form. Here, you know the architecture of the network, and you can do backprop by hand; just turn the calculations that you do by hand into code and return the result.

Hint 2: Before writing the code, try running backprop by hand on the input given in the problem. This way you can more easily debug your code by printing out the $\partial H / \partial z$'s and $\partial H / \partial a$'s and comparing them with what you got by hand.

Hint 3: When your code is written correctly, the first entry of your result should be 15, and the entries should sum to 55.

- c) Let's train our network using the square loss. The empirical risk with respect to the square loss is:

$$R(\vec{w}) = \frac{1}{n} \sum_{i=1}^n (H(x^{(i)}; \vec{w}) - y_i)^2,$$

where the summation ranges over the training data.

We will train the network using gradient descent, meaning that we need the gradient of the risk, $\nabla R(\vec{w})$. In symbols, this is:

$$\nabla R(\vec{w}) = \frac{2}{n} \sum_{i=1}^n (H(x^{(i)}; \vec{w}) - y_i) \nabla H(x^{(i)}, \vec{w}).$$

Write a function, `del_R(w)` which takes in a parameter vector `w` and returns the gradient at `w`, $\nabla R(\vec{w})$, as a numpy array with 7 entries. Use your function to compute $\nabla R((1, 2, -1, -2, 3, 4, -5)^T)$. Show your code.

Hint 1: Notice that, to compute the gradient, you'll need to sum over the 50 points in the training data provided above, but the training data is not an argument to `del_R`. That's OK: either store the training data in a global variable, or, if you'd prefer to write nicer code, write a higher-order function `make_del_R(data)` which accepts the training data and returns `del_R` as a closure.

Hint 2: When implemented correctly, your function should return an array whose elements sum to around 2135. If you get something around 2148 instead, make sure you're not using the ReLU on the output node.

- d) Run gradient descent to train your neural network on the data. Plot your trained $H(x)$ in the interval $[-10, 10]$, on top of a scatter plot of the training data; it should fit the data reasonably well.

Hint 1: Code for gradient descent was provided in lecture. Don't worry about using *stochastic* gradient descent (though you can if you'd like...).

Hint 2: To run gradient descent, you will need to choose an initial parameter vector, a stopping criterion (threshold), and a learning rate. Try setting the initial parameter vector randomly using `np.random.uniform(0, 1, 7)`. The learning rate should be pretty small. Modify the gradient descent

code to print `np.linalg.norm(x - x_new)` on every iteration; if it doesn't look like this is converging to zero, you picked too big of a step size. If gradient descent takes more than a minute or two to finish, you either set the learning rate incorrectly, or your threshold is too high.

Hint 3: Your neural network will likely not fit the data well on the first try; use your plot to determine whether or not this is the case. Remember, the neural network objective function is very non-convex, and gradient descent will likely get stuck in a local minimum. You will probably need to try a few random initializations of the parameter vector.

Hint 4: It can be annoying to find an initialization of the parameter vector that works just to lose it by re-running the cell on accident. You avoid this by using a random “seed”. For example:

```
np.random.seed(42)
w_init = np.random.uniform(0, 1, 7)
```

Here, 42 is the “seed”. Using a seed will guarantee that `np.random.uniform` returns the same sequence of “random” numbers. Changing the seed to, say, 42, changes the sequence.

Suggestion 1: Once you're done, take a moment to appreciate how much easier it is to train, for example, a Gaussian RBF network, or how easy `pytorch` makes this process.